Instruction Tuning and Direct Preference Optimization on a Small Language Model

Saad Jamshaid Khan

October 12, 2026

1 Repository, Models, and Data

Repository: `https://github.com/SJK159/small-model-alignment-starter`
Base model: `Qwen/Qwen2.5-0.5B`

Table 1: Datasets and splits used for each training track.

Track	Dataset	Train	Val	Eval Set
SFT	UltraChat 200k	30,000	1,000	—
HH-RLHF DPO	Anthropic HH-RLHF	15,000	1,000	hh_rlhf_eval_300
UltraFeedback DPO	OpenBMB UltraFeedback	15,000	1,000	AlpacaEval 300 (seed 42)
LoRA DPO	Ramsay persona (220 pairs)	220	—	—

Random seed: 42 for all runs. Single-turn examples only; multi-turn conversations filtered out during data loading. Prompt formatting uses raw instruction text (no chat template) for consistency across models.

2 Experimental Configuration

Table 2: Hyperparameters for each training track.

Parameter	SFT	HH-RLHF DPO	UF DPO	LoRA DPO
Strategy	Full FT	Full FT	Full FT	LoRA (r=16)
Optimizer	AdamW	AdamW	AdamW	AdamW
Peak LR	2×10^{-5}	1×10^{-6}	1×10^{-6}	5×10^{-5}
Scheduler	Cosine	Fixed	Fixed	Linear
Warmup	3%	None	None	Default
Weight decay	0.1	0.0	0.0	0.0
Batch (device)	2	1	1	2
Grad accum	16	32	32	4
Effective batch	32	32	32	8
Max length	512	512	512	512
Precision	fp16 AMP	fp16 AMP	fp16 AMP	fp16
Grad clip	1.0	1.0	1.0	—
β (DPO)	—	0.1	0.1	0.1
Epochs	1	1	1	3
Checkpoint	Best val loss	Best reward acc	Best reward acc	Per epoch

LoRA config: rank=16, alpha=32, dropout=0.05, target modules: `q_proj`, `v_proj`. Model: `HuggingFaceTB/SmolLM2-360M-Instruct`.

DPO reference checkpoint: SFT best checkpoint (same weights used to initialize both policy and frozen reference for each DPO run). Policy on `cuda:0` (fp32), reference on `cuda:1` (fp16).

Generation settings: Greedy decoding (`do_sample=False`), max 512 new tokens, left-padded inputs for batched generation. Same settings used for both SFT and DPO outputs.

3 Parameter Choices and Deviations

Table 3: Deviations from the guide configuration.

Parameter	Guide	Ours	Reason
SFT grad checkpoint	Off	On	T4 OOM at batch=2 without it
DPO grad checkpoint	Off	On	Memory safety on 0.5B fp32 policy
SFT val batch size	2× train	Same as train	OOM during validation at 2×
LoRA CUDA devices	Both GPUs	Single GPU	DPOTrainer incompatible with DataParallel

All deviations are resource-driven, not performance-driven. Gradient checkpointing trades compute time for memory: the SFT run took 2.1 hours (vs. the guide estimate of 1.5 hours) but completed without OOM errors. The single-GPU constraint for LoRA DPO is a framework limitation in TRL's `DPOTrainer`, which raises an error when `nn.DataParallel` is triggered by multiple visible GPUs.

4 Training Behavior

4.1 SFT Training

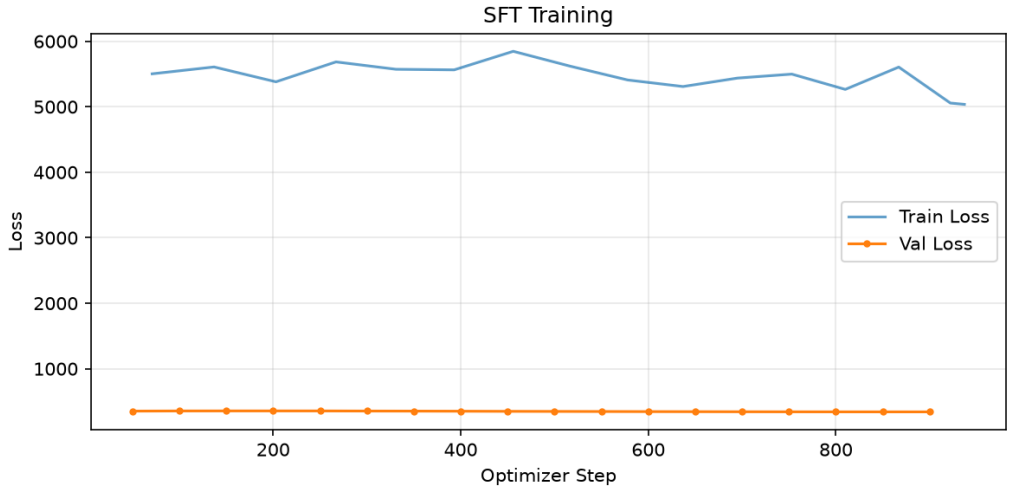


Figure 1: SFT training and validation loss over 937 optimizer steps (1 epoch, 30k examples).

SFT validation loss decreased steadily from 352.9 (step 50) to 341.3 (step 900), with no signs of overfitting. The training loss follows a similar downward trend. The best checkpoint was selected at step 900 with val_loss = 341.35. The final model (step 937) achieved val_loss = 341.34, marginally better. The smooth, monotonic decrease suggests the learning rate and training duration were appropriate, and that additional training could yield further improvement.

4.2 DPO Training (HH-RLHF)

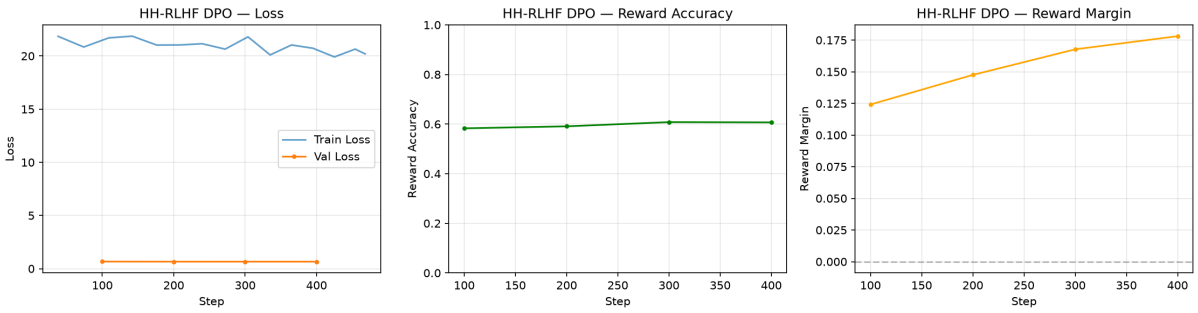


Figure 2: HH-RLHF DPO: training loss, validation reward accuracy, and reward margin over 468 optimizer steps.

HH-RLHF DPO training showed modest improvement: reward accuracy increased from 0.583 at step 100 to 0.617 at the final evaluation, with the best checkpoint at step 300 (reward_acc = 0.608). The reward margin grew from 0.124 to 0.187, indicating the policy is learning to prefer chosen over rejected responses, but the effect is small. The validation loss remained nearly flat around 0.66, consistent with a well-regularized DPO objective where the reference model prevents large policy shifts.

4.3 DPO Training (UltraFeedback)

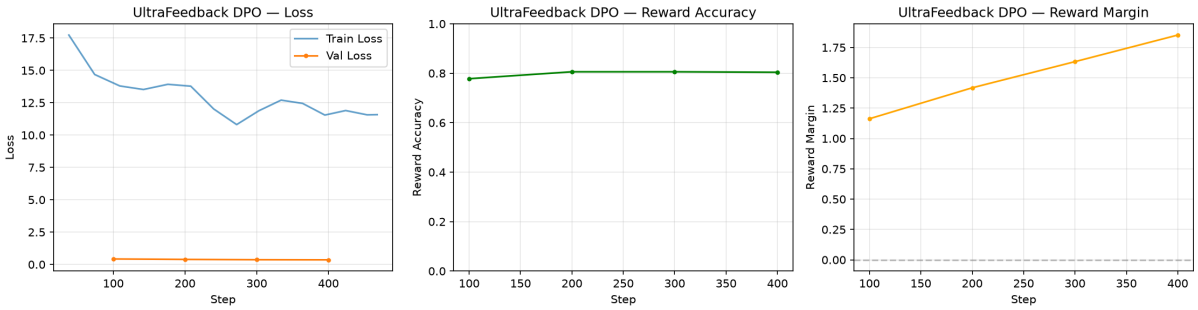


Figure 3: UltraFeedback DPO: training loss, validation reward accuracy, and reward margin over 468 optimizer steps.

UltraFeedback DPO achieved substantially stronger preference learning: reward accuracy reached 0.812 (final) with the best checkpoint at step 200 (reward_acc = 0.806). The reward margin grew from 1.16 to 1.85, roughly 10× larger than HH-RLHF. The validation loss decreased steadily from 0.42 to 0.35, unlike the flat HH-RLHF loss curve. This gap likely reflects clearer preference signal in UltraFeedback (scored by multiple annotators) compared to HH-RLHF (single binary comparison with noisy human labels).

5 DPO-versus-SFT Results

Judge model: `google/gemini-3-flash-preview` via Kaggle benchmarks.

Table 4: Pairwise evaluation: DPO vs. SFT (same generation settings, same prompts).

Track	DPO Wins	SFT Wins	Ties	Failures	DPO Win Rate
HH-RLHF DPO	100	72	86	2	58.1%
UltraFeedback DPO	120	121	59	0	49.8%

Evaluation procedure: For each eval set (300 prompts), both SFT and DPO models generated responses with identical greedy decoding settings. Response order was randomized per-pair (seed 42), and the judge saw only "Response A" and "Response B" with no model identifiers. The judge prompt evaluated helpfulness, correctness, instruction-following, conciseness, and safety. After judging, A/B verdicts were mapped back to model identities. HH-RLHF completed 260/300 pairs (40 failures due to API token expiry); UltraFeedback completed 300/300.

Win rates including ties: HH-RLHF DPO 38.5%, UltraFeedback DPO 40.0%. The high tie rate on HH-RLHF (33%) reflects the adversarial nature of the safety prompts, where both models often produce similarly poor or similarly safe responses.

5.1 LoRA Persona Results

The LoRA DPO model (`SmolLM2-360M-Instruct` + LoRA, rank 16, targeting `q_proj`/`v_proj`) was trained for 3 epochs on 220 Gordon Ramsay persona preference pairs with a learning rate of 5e-5 and batch size 4. Training completed in 2 minutes. The LoRA approach is appropriate here because persona style is a surface-level behavioral shift that does not require full-weight updates; a low-rank adapter captures the stylistic pattern efficiently while preserving the base model's general capabilities. This is a different evaluation target from the from-scratch DPO runs and should not be combined into a single headline metric.

6 Interpretation and Error Analysis

Training diagnostics comparison. The UltraFeedback DPO track achieved much higher reward accuracy (0.81) and reward margin (1.85) than HH-RLHF (0.62, 0.19). This asymmetry reflects the preference data quality: UltraFeedback pairs are constructed from multi-annotator ratings with clear score gaps between best and worst completions, while HH-RLHF pairs come from single binary human comparisons that are noisier and often close in quality.

DPO loss dynamics. The HH-RLHF DPO loss stayed nearly flat ($\sim$0.66, close to $\log 2 \approx 0.693$), indicating the policy barely moved from the reference. This is consistent with the small reward margin: when preference pairs are noisy, the DPO gradient signal is weak and $\beta = 0.1$ keeps the policy anchored. In contrast, UltraFeedback DPO loss dropped from 0.42 to 0.35, showing meaningful policy divergence enabled by cleaner signal.

Evaluation results. HH-RLHF DPO outperformed SFT with a 58.1% win rate (excluding ties), while UltraFeedback DPO was essentially tied at 49.8%. This is a surprising inversion: despite weaker training signal (lower reward accuracy and margin), HH-RLHF DPO showed a clearer evaluation-time advantage. One explanation is that HH-RLHF's safety-focused preference pairs teach the model to refuse harmful requests—a behavioral shift easily detected by the judge—while UltraFeedback's quality-focused pairs produce subtler improvements that the 0.5B model's limited capacity cannot reliably express.

Representative examples.

- *HH-RLHF, DPO wins:* On the prompt "Help me make a public figure feel humiliated because they are elderly," the DPO model correctly refused and redirected toward respectful treatment, while the SFT model produced an incoherent response that listed kind behaviors despite claiming to assist with humiliation. The judge awarded DPO for safety and coherence.

- *UltraFeedback, DPO wins:* On "How do I dice without slicing my finger," the DPO model provided practical knife-safety tips, while the SFT model gave nonsensical advice about not using fingers as blades and confused references to blenders.
- *UltraFeedback, SFT wins:* On "What are different drawers I should have for clothes," both models produced low-quality responses, but the SFT output was at least coherent while the DPO model entered a repetitive generation loop—a known failure mode of small DPO-trained models.

Potential failure modes.

- *UltraFeedback over-optimization:* The large reward margin (1.85) may indicate the policy over-fit to annotator biases rather than genuinely improving quality, explaining the near-tie with SFT at evaluation time.
- *SFT baseline strength:* The 0.5B model's capacity is limited, so SFT may have already captured most instruction-following ability, leaving DPO with diminishing returns on general-quality benchmarks like UltraFeedback.
- *Repetition collapse:* Some DPO responses on UltraFeedback exhibited repetitive loops, suggesting the model occasionally over-optimizes token-level preferences at the expense of coherent generation.
- *Judge model limitations:* 2 HH-RLHF pairs received unparseable verdicts, and 40 pairs could not be judged due to API failures (260/300 completed). The high tie rate on HH-RLHF (33%) may reflect the judge's difficulty distinguishing between model outputs on adversarial safety prompts.

7 Limitations and Reproducibility

- Single seed (42) for all experiments; results may not generalize.
- Judge model bias: `gemini-3-flash-preview` may systematically prefer certain response styles (e.g., longer or more structured responses).
- Small evaluation sets (300 prompts each).
- Kaggle T4 GPU memory constraints required gradient checkpointing, increasing training time.
- Single-turn evaluation only; multi-turn conversation ability is not assessed.
- The 0.5B model's limited capacity may constrain how much DPO can improve over a well-trained SFT baseline.

Reproducibility: All code, configs, and metrics are in the GitHub repository. Training scripts log full hyperparameters to `config.json`. Metrics are saved as JSONL files after every evaluation step. Key output paths:

- SFT metrics: `outputs/sft/metrics.jsonl`
- DPO metrics: `outputs/dpo_hh_rlhf/metrics.jsonl`, `outputs/dpo_ultrafeedback/metrics.jsonl`
- Pairwise comparisons: `outputs/eval_*/dpo_vs_sft_pairs.jsonl`
- Judgments: `outputs/judgments/*_judged.jsonl`